Signzy SDE Internship Assignment Submission

GitHub Repository: https://github.com/yashwanth04112005/signzy

Source code

The complete implementation consists of an Express backend API, a React client dashboard, and a MongoDB persistence tier. Key parts of the core implementation are highlighted below:

MongoDB Models (`server/db.js`)

Manages persistence for registered vendors, live rate-limit request sliding-windows, and auto-purging 30-day TTL log collections.

```
import mongoose from "mongoose";

const metricsSchema = new mongoose.Schema(
  {
    requestCount:       { type: Number, default: 0 },
    successCount:       { type: Number, default: 0 },
    failureCount:       { type: Number, default: 0 },
    consecutiveFailures:{ type: Number, default: 0 },
    totalLatencyMs:     { type: Number, default: 0 },
    lastLatencyMs:      { type: Number, default: 0 },
    lastRequestAt:      { type: Date,   default: null }
  },
  { _id: false }
);

const vendorSchema = new mongoose.Schema(
  {
    name:                   { type: String, required: true },
    capability:             { type: String, required: true },
    weight:                 { type: Number, default: 50 },
    costPerRequest:         { type: Number, default: 1.0 },
    timeoutMs:              { type: Number, default: 2500 },
    rateLimitPerMinute:     { type: Number, default: 100 },
    priority:               { type: Number, default: 1 },
    supportedFeatures:      { type: [String], default: [] },
    strategy:               { type: String, default: "weighted" },
    baseLatencyMs:          { type: Number, default: 900 },
    enabled:                { type: Boolean, default: true },
    status:                 { type: String, enum: ["UP", "DOWN"], default: "UP" },
    metrics:                { type: metricsSchema, default: () => ({}) },
    recentRequestTimestamps:{ type: [Number], default: [] }
  },
  { timestamps: { createdAt: "createdAt", updatedAt: false } }
);
```

```
vendorSchema.index({ name: 1, capability: 1 }, { unique: true });
export const Vendor = mongoose.model("Vendor", vendorSchema);
```

Core Strategy Sorting Handler (`server/routing.js`)

Evaluates vendor candidates against constraints and sorts them according to the selected strategy.

```
function compareByStrategy(strategy, left, right, metricsMap, request) {
  switch (strategy) {
    case "lowest-latency":
      return (left.baseLatencyMs ?? 0) - (right.baseLatencyMs ?? 0);
    case "lowest-cost":
      return (left.costPerRequest ?? 0) - (right.costPerRequest ?? 0);
    case "priority":
      // Priority-based routing: select the vendor with the highest priority
configuration (lowest number)
      return (left.priority ?? 0) - (right.priority ?? 0);
    case "failover":
      // Failover routing: establishes the sequence of secondary/backup attempts
if the primary fails
      return (left.priority ?? 0) - (right.priority ?? 0);
    case "feature-based":
      return (right.supportedFeatures?.length ?? 0) -
(left.supportedFeatures?.length ?? 0);
    case "health-based": {
      // Health-based routing: sorts strictly by live success rate (descending)
and error rate (ascending)
      const leftMv = metricsMap.get(left.id ?? left._id?.toString());
      const rightMv = metricsMap.get(right.id ?? right._id?.toString());
      if (!leftMv && !rightMv) return 0;
      if (!leftMv) return 1;
      if (!rightMv) return -1;
      if (leftMv.successRate !== rightMv.successRate) {
        return (rightMv.successRate ?? 0) - (leftMv.successRate ?? 0);
      }
      return (leftMv.errorRate ?? 0) - (rightMv.errorRate ?? 0);
    }
    case "weighted":
    default:
      return (
        getVendorScore(right, metricsMap.get(right.id ?? right._id?.toString()),
request) -
        getVendorScore(left,  metricsMap.get(left.id  ?? left._id?.toString()),
request)
      );
  }
}
```

README

Run Locally

Prerequisites: Node.js 18+ and MongoDB running locally on `mongodb://localhost:27017/signzy-vendor-router`.

```
# Install dependencies for both frontend and backend workspace
npm install

# Run the development environment concurrently (both backend on port 3000 and
client on port 5173)
npm run dev
```

- **Backend Gateway:** `http://localhost:3000`
- **Client Dashboard:** `http://localhost:5173`

```
# Run the test suite (covers all strategy, validation, round-robin, and override
logic)
npm test
```

Core Features

- **8 routing strategies** — weighted, priority, lowest-latency, lowest-cost, failover, round-robin, feature-based, health-based.
- **Auto-switch** when a vendor is DOWN, rate-limited, too slow, missing required features, or has a live error rate > 40%.
- **Agentic AI Support** — plain-English routing config parser, strategy recommendation, unhealthy vendor detection, and fallback rule generator on the frontend.
- **Full CRUD vendor management** via REST API.

Sample vendor configs

The database seeds 3 default vendors on first run:

1. VendorA (Primary / High Traffic)

```
{
  "name": "VendorA",
  "capability": "PAN_VERIFICATION",
  "weight": 70,
  "costPerRequest": 1.5,
  "timeoutMs": 2000,
  "rateLimitPerMinute": 100,
  "priority": 1,
  "supportedFeatures": ["name-match", "pan-status"],
  "strategy": "weighted",
  "baseLatencyMs": 1200,
```

```
  "enabled": true,
  "status": "UP"
}
```

2. VendorB (Secondary / Fast & Custom Features)

```
{
  "name": "VendorB",
  "capability": "PAN_VERIFICATION",
  "weight": 30,
  "costPerRequest": 1.2,
  "timeoutMs": 3000,
  "rateLimitPerMinute": 50,
  "priority": 2,
  "supportedFeatures": ["name-match", "pan-status", "low-cost"],
  "strategy": "priority",
  "baseLatencyMs": 850,
  "enabled": true,
  "status": "UP"
}
```

3. VendorC (Tertiary / Low Cost Backup)

```
{
  "name": "VendorC",
  "capability": "PAN_VERIFICATION",
  "weight": 50,
  "costPerRequest": 1.1,
  "timeoutMs": 2500,
  "rateLimitPerMinute": 80,
  "priority": 3,
  "supportedFeatures": ["name-match", "pan-status"],
  "strategy": "health-based",
  "baseLatencyMs": 1000,
  "enabled": true,
  "status": "UP"
}
```

Sample API Requests/Responses

Route Request (POST `/route`)

Request Payload:

```
{
  "capability": "PAN_VERIFICATION",
  "payload": {
    "pan": "ABCDE1234F",
    "name": "Rahul Sharma"
  },
  "requirements": {
    "strategy": "lowest-cost",
    "requiredFeatures": ["name-match"]
  }
}
```

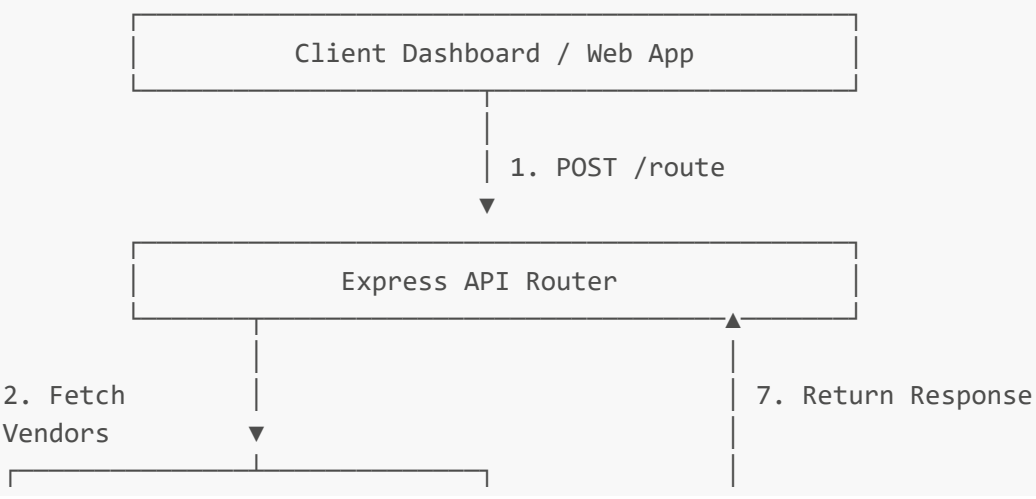
Response Payload:

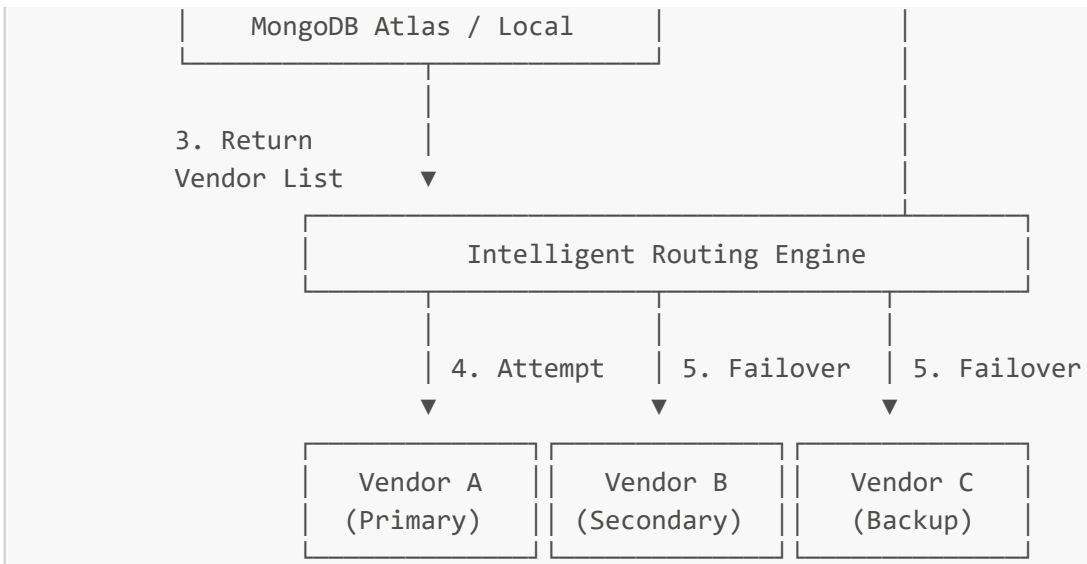
```
{
  "status": "SUCCESS",
  "vendorUsed": "VendorC",
  "routingReason": "VendorC was selected for lowest cost (₹1.1).",
  "latencyMs": 1024,
  "cost": 1.1,
  "response": {
    "panStatus": "VALID",
    "nameMatch": true,
    "referenceId": "f7a30b7a-8fbb-4e98-b80c-c6cfd88d227b"
  }
}
```

Architecture diagram

System Architecture

This diagram groups the platform components into logical layers (Client Dashboard, Platform Backend, Database Persistence, and External Vendors):





Routing Decision Engine Flow

Maps the exact step-by-step logic the routing engine executes to filter, rank, and failover across candidates:

```
        ├── Success ──▶ [Return Success Response & Update Live Metrics]
        │
        └── Failure ──▶ [Log fallback reason, attempt next candidate]
                          └── If all candidates fail: Return Failure envelope
```

Explanation of routing decisions

The routing engine dynamically evaluates and ranks eligible candidates using one of **8 strategies** depending on configuration or request requirement flags:

Strategy	Description	Selection Metric
`weighted`	Scores candidates based on composite formula: latency (40%), success rate (30%), cost (20%), and availability (10%) combined with vendor weight.	Composite scoring algorithm
`priority`	Selects candidate strictly based on the configured priority number (lowest priority value first).	Ascending priority ordering
`lowest-latency`	Selects candidate with the lowest average baseline latency to minimize execution delay.	Ascending latency ordering
`lowest-cost`	Selects candidate with the lowest cost per call to maximize margin. Also triggered when `preferLowCost` flag is true.	Ascending cost ordering
`failover`	Systematically falls over to the next priority vendor candidate if the primary fails or times out.	Failover attempt retry sequence
`round-robin`	Rotates requests across eligible vendors using an atomic database counter per capability.	Persistent modular counter index
`feature-based`	Selects vendor candidates supporting the highest number of overall features first.	Descending supported features length
`health-based`	Selects vendor candidates strictly ordered by success rate (descending) and error rate (ascending).	Live performance metrics ranking

---

AI_USAGE.md if AI tools are used

I used an AI assistant selectively to speed up standard boilerplate creation and formatting tasks:

- **Boilerplate Schemas:** I had the assistant generate standard Mongoose models based on my database schema definitions.
- **Frontend CSS Adjustments:** I used the assistant to help write basic CSS styling selectors and load the Inter Google Font.
- **Documentation Formatting:** I used the assistant to format my README API tables and cleanup my text flowcharts.